

ST5209/X Assignment 4

Jiao Dian

Submission

1. Render the document to get a .pdf printout.
2. Submit both the .qmd and .pdf files to Canvas.

1. AR polynomial

Consider the AR(2) model

$$X_t = 4 + 0.5X_{t-1} - 0.25X_{t-2} + W_t. \quad (1)$$

a. Autoregressive polynomial:

The AR polynomial is obtained by writing the model as $X_t - 0.5X_{t-1} + 0.25X_{t-2} = 4 + W_t$, giving

$$\phi(z) = 1 - 0.5z + 0.25z^2.$$

b. Roots:

We solve $\phi(z) = 0$, i.e. $0.25z^2 - 0.5z + 1 = 0$. Multiplying by 4: $z^2 - 2z + 4 = 0$.

$$z = \frac{2 \pm \sqrt{4 - 16}}{2} = \frac{2 \pm \sqrt{-12}}{2} = 1 \pm i\sqrt{3}.$$

```
polyroot(c(1, -0.5, 0.25))
```

```
[1] 1+1.732051i 1-1.732051i
```

The roots are $z_1 = 1 + i\sqrt{3}$ and $z_2 = 1 - i\sqrt{3}$.

c. Causality:

The modulus of each root is $|z| = \sqrt{1^2 + (\sqrt{3})^2} = \sqrt{4} = 2 > 1$. Since both roots lie strictly outside the unit circle, the model is **causal**.

d. **Linear process coefficients** $\psi_0, \psi_1, \psi_2, \psi_3$:

Following the strategy in `linear_process_coefs.qmd`, we use the relation $\phi(z) \sum_{j=0}^{\infty} \psi_j z^j = 1$. Writing $\phi_0 = 1$, $\phi_1 = -0.5$, $\phi_2 = 0.25$, the recursive formula is $\sum_{j=0}^2 \phi_j \psi_{k-j} = 0$ for $k \geq 1$, i.e. $\psi_k = 0.5\psi_{k-1} - 0.25\psi_{k-2}$, with $\psi_0 = 1$ and $\psi_j = 0$ for $j < 0$.

- $\psi_0 = 1$
- $k = 1$: $\psi_1 + (-0.5)(1) = 0 \Rightarrow \psi_1 = 0.5$
- $k = 2$: $\psi_2 + (-0.5)(0.5) + (0.25)(1) = 0 \Rightarrow \psi_2 = 0.25 - 0.25 = 0$
- $k = 3$: $\psi_3 + (-0.5)(0) + (0.25)(0.5) = 0 \Rightarrow \psi_3 = -0.125$

Verification using R:

```
ARMAtoMA(ar = c(0.5, -0.25), lag.max = 3)
```

```
[1] 0.500 0.000 -0.125
```

So $\psi_0 = 1$, $\psi_1 = 0.5$, $\psi_2 = 0$, $\psi_3 = -0.125$.

The mean is $\mu = \frac{4}{1-0.5+0.25} = \frac{4}{0.75} = \frac{16}{3}$.

2. Likelihood

Consider the AR(1) model

$$X_t = 1 - 0.6X_{t-1} + W_t,$$

where $W_t \sim WN(0, 0.25)$ is Gaussian white noise. Observations: $x_1 = 0.2, x_2 = -0.3, x_3 = 0.4$.

a. **Mean of X_t :**

Writing the model as $X_t = c + \phi X_{t-1} + W_t$ with $c = 1$ and $\phi = -0.6$, the mean is

$$\mu = \frac{c}{1 - \phi} = \frac{1}{1 - (-0.6)} = \frac{1}{1.6} = 0.625.$$

b. **Conditional likelihood given x_1 :**

Conditioning on x_1 , the conditional likelihood is $L = p(x_2|x_1) \cdot p(x_3|x_2)$. Since $X_t|X_{t-1} = x_{t-1} \sim N(1 - 0.6x_{t-1}, 0.25)$, each factor is a Gaussian density.

The conditional means are:

- $\mu_2 = 1 - 0.6(0.2) = 0.88$
- $\mu_3 = 1 - 0.6(-0.3) = 1.18$

The residuals are:

- $e_2 = x_2 - \mu_2 = -0.3 - 0.88 = -1.18$
- $e_3 = x_3 - \mu_3 = 0.4 - 1.18 = -0.78$

The conditional likelihood is:

$$L = \prod_{t=2}^3 \frac{1}{\sqrt{2\pi \cdot 0.25}} \exp\left(-\frac{e_t^2}{2 \cdot 0.25}\right) = \frac{1}{2\pi \cdot 0.25} \exp\left(-\frac{(-1.18)^2 + (-0.78)^2}{0.5}\right)$$

```
e2 <- -0.3 - (1 - 0.6 * 0.2)
e3 <- 0.4 - (1 - 0.6 * (-0.3))
sigma2 <- 0.25

cat("e2 =", e2, "\n")
```

e2 = -1.18

```
cat("e3 =", e3, "\n")
```

e3 = -0.78

```
cat("e2^2 + e3^2 =", e2^2 + e3^2, "\n")
```

e2^2 + e3^2 = 2.0008

```
L <- prod(dnorm(c(-0.3, 0.4), mean = c(1 - 0.6 * 0.2, 1 - 0.6 * (-0.3)), sd = sqrt(sigma2)))
cat("Conditional likelihood:", L, "\n")
```

Conditional likelihood: 0.01164146

The sum of squared residuals is $(-1.18)^2 + (-0.78)^2 = 1.3924 + 0.6084 = 2.0008$, so the conditional likelihood evaluates to approximately 0.0116. This small value reflects the fact that the observed data points are relatively far from their conditional means, suggesting a poor fit for these particular observations.

3. Reversibility

- a. Simulate a sample trajectory of length $n = 500$ from the AR(2) model in Problem 1.

```
set.seed(5209)
xt <- arima.sim(model = list(ar = c(0.5, -0.25)), n = 500)
```

- b. Reverse the time index.

```
xt_rev <- rev(as.numeric(xt))
```

- c. Fit an AR(2) model to the reversed time series.

```
rev_tsibble <- tsibble(
  t = 1:500,
  X = xt_rev,
  index = t
)

rev_fit <- rev_tsibble |>
  model(AR(X ~ order(2)))
```

- d. Inspect the model parameters.

```
rev_fit |> tidy()
```

```
# A tibble: 3 x 6
  .model      term      estimate std.error statistic  p.value
  <chr>      <chr>      <dbl>    <dbl>    <dbl>    <dbl>
1 AR(X ~ order(2)) constant  0.0596  0.0418     1.43 1.54e- 1
2 AR(X ~ order(2)) ar1       0.499   0.0433    11.5 2.48e-27
3 AR(X ~ order(2)) ar2      -0.239   0.0436    -5.49 6.54e- 8
```

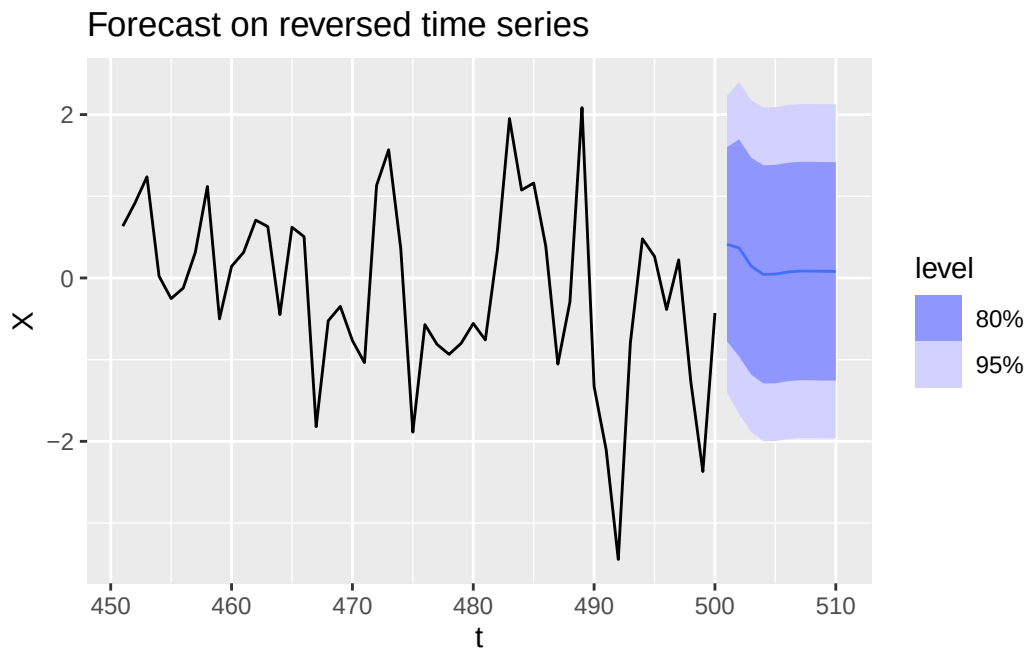
The fitted AR(2) coefficients are approximately $\hat{\phi}_1 \approx 0.499$ and $\hat{\phi}_2 \approx -0.239$, which are close to the true values $\phi_1 = 0.5$ and $\phi_2 = -0.25$ from Equation 1. This is because of the **reversibility** property: for a causal stationary process, the autocovariance function $\gamma(h)$ is symmetric ($\gamma(h) = \gamma(-h)$), so the Yule-Walker equations are identical whether we read the time series forward or backward. The small discrepancies from the true values are due to finite-sample estimation variability.

- e. Forecast with $h = 10$.

```
rev_fit |>
  forecast(h = 10)
```

```
# A fable: 10 x 4 [1]
# Key:   .model [1]
   .model      t
   <chr>      <dbl>
1 AR(X ~ order(2)) 501
2 AR(X ~ order(2)) 502
3 AR(X ~ order(2)) 503
4 AR(X ~ order(2)) 504
5 AR(X ~ order(2)) 505
6 AR(X ~ order(2)) 506
7 AR(X ~ order(2)) 507
8 AR(X ~ order(2)) 508
9 AR(X ~ order(2)) 509
10 AR(X ~ order(2)) 510
# i 2 more variables: X <dist>, .mean <dbl>
```

```
rev_fit |>
  forecast(h = 10) |>
  autoplot(rev_tsibble |> filter(t > 450)) +
  labs(title = "Forecast on reversed time series")
```



Since we reversed the time series, forecasting forward on the reversed series corresponds to **backcasting** — predicting values that would have occurred before the start of the original time series. In the original time series, the observations are indexed $t = 1, 2, \dots, 500$. After reversal, time $t = 1$ in the reversed series corresponds to $t = 500$ in the original, so forecasting at $t = 501, \dots, 510$ in the reversed series corresponds to predicting $X_0, X_{-1}, \dots, X_{-9}$ in the original. The forecasts converge toward the process mean $\mu = 0$ as the forecast horizon increases, which is expected since `arima.sim()` generates a zero-mean AR(2) process (without the constant term 4) and the model is causal and stationary.

4. Yule-Walker

- a. **Yule-Walker equations** for the AR(2) model $X_t = 0.5X_{t-1} - 0.25X_{t-2} + W_t$ (ignoring the constant since it doesn't affect autocovariances):

Multiplying both sides by X_{t-h} and taking expectations:

- $h = 0$: $\gamma(0) = 0.5\gamma(1) - 0.25\gamma(2) + \sigma^2$
- $h = 1$: $\gamma(1) = 0.5\gamma(0) - 0.25\gamma(1)$
- $h = 2$: $\gamma(2) = 0.5\gamma(1) - 0.25\gamma(0)$

- b. **Matrix form.** Rearranging:

$$\begin{bmatrix} 1 & -0.5 & 0.25 \\ -0.5 & 1.25 & 0 \\ 0.25 & -0.5 & 1 \end{bmatrix} \begin{bmatrix} \gamma(0) \\ \gamma(1) \\ \gamma(2) \end{bmatrix} = \begin{bmatrix} \sigma^2 \\ 0 \\ 0 \end{bmatrix}.$$

Row derivations:

- Row 1: $\gamma(0) - 0.5\gamma(1) + 0.25\gamma(2) = \sigma^2$
- Row 2: $-0.5\gamma(0) + (1 + 0.25)\gamma(1) + 0 \cdot \gamma(2) = 0$ (from $\gamma(1) = 0.5\gamma(0) - 0.25\gamma(1)$)
- Row 3: $0.25\gamma(0) - 0.5\gamma(1) + \gamma(2) = 0$ (from $\gamma(2) = 0.5\gamma(1) - 0.25\gamma(0)$)

- c. **Solve numerically** (assuming $\sigma^2 = 1$):

```
A <- matrix(c(1, -0.5, 0.25,
              -0.5, 1.25, 0,
              0.25, -0.5, 1), nrow = 3, byrow = TRUE)
gamma_vals <- solve(A, c(1, 0, 0))
names(gamma_vals) <- c("gamma(0)", "gamma(1)", "gamma(2)")
gamma_vals
```

```
      gamma(0)      gamma(1)      gamma(2)
1.26984127  0.50793651 -0.06349206
```

We obtain $\gamma(0) \approx 1.270$, $\gamma(1) \approx 0.508$, and $\gamma(2) \approx -0.063$. Note that $\gamma(2) < 0$, which is consistent with the negative AR(2) coefficient $\phi_2 = -0.25$.

d. **Top left entry of Γ_2^{-1} :**

For a 2×2 matrix $\Gamma_2 = \begin{bmatrix} \gamma(0) & \gamma(1) \\ \gamma(1) & \gamma(0) \end{bmatrix}$, the inverse is $\frac{1}{\gamma(0)^2 - \gamma(1)^2} \begin{bmatrix} \gamma(0) & -\gamma(1) \\ -\gamma(1) & \gamma(0) \end{bmatrix}$.

The top left entry is:

$$[\Gamma_2^{-1}]_{1,1} = \frac{\gamma(0)}{\gamma(0)^2 - \gamma(1)^2}.$$

```
g0 <- gamma_vals[1]
g1 <- gamma_vals[2]
top_left_inv <- g0 / (g0^2 - g1^2)
cat("Top left entry of Gamma_2 inverse:", top_left_inv, "\n")
```

Top left entry of Gamma_2 inverse: 0.9375

The top left entry is $\frac{\gamma(0)}{\gamma(0)^2 - \gamma(1)^2} \approx \frac{1.270}{1.270^2 - 0.508^2} = \frac{1.270}{1.354} \approx 0.9375$.

Verification using `solve()`:

```
Gamma2 <- matrix(c(g0, g1, g1, g0), nrow = 2)
solve(Gamma2)
```

```
      [,1]      [,2]
[1,] 0.9375 -0.3750
[2,] -0.3750 0.9375
```

The top left entry of Γ_2^{-1} is indeed 0.9375, confirming the formula.

e. **95% confidence interval for ϕ_1 .**

The asymptotic distribution of the Yule-Walker estimator is $\hat{\phi} \approx N(\phi, \frac{\sigma^2}{n} \Gamma_2^{-1})$. Using $n = 500$ from Problem 3 and the fitted parameters:

```
n <- 500
sigma2_hat <- 1 # white noise variance
se_phi1 <- sqrt(sigma2_hat * top_left_inv / n)
cat("Standard error of phi1_hat:", round(se_phi1, 4), "\n")
```

Standard error of phi1_hat: 0.0433

```
phi1_hat <- rev_fit |> tidy() |> filter(term == "ar1") |> pull(estimate)
ci_lower <- phi1_hat - 1.96 * se_phi1
ci_upper <- phi1_hat + 1.96 * se_phi1
cat("phi1_hat:", round(phi1_hat, 4), "\n")
```

```
phi1_hat: 0.4987
```

```
cat("95% CI for phi_1: [", round(ci_lower, 4), ",", round(ci_upper, 4), "]\n")
```

```
95% CI for phi_1: [ 0.4139 , 0.5836 ]
```

The standard error is $\sqrt{\sigma^2[\Gamma_2^{-1}]_{1,1}/n} = \sqrt{0.9375/500} \approx 0.0433$. Using the estimated $\hat{\phi}_1 \approx 0.499$ from Problem 3d, the 95% confidence interval is approximately $[0.414, 0.584]$. This interval contains the true value $\phi_1 = 0.5$, as expected.

5. Recursive forecasting for ARMA(1, 1)

Consider the ARMA(1,1) model $X_t - 0.5X_{t-1} = W_t + 0.5W_{t-1}$.

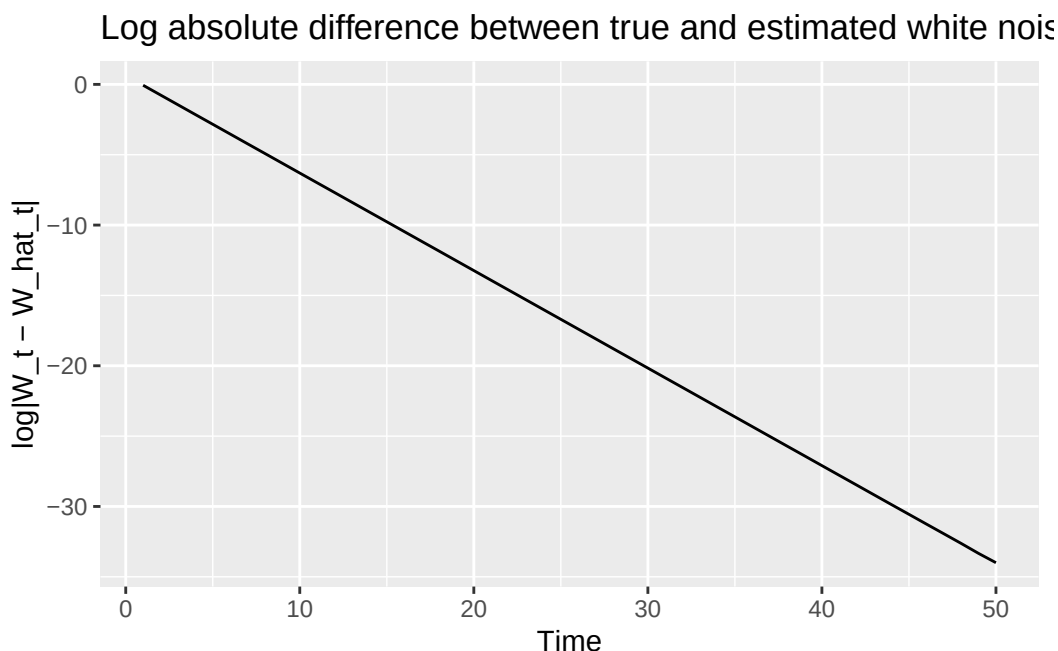
```
set.seed(5209)
n <- 50
wn <- rnorm(n)
xt <- arima.sim(model = list(ar = 0.5, ma = 0.5), innov = wn, n = n)
```

- a. **Truncated white noise estimation.** From the model, $W_t = X_t - 0.5X_{t-1} - 0.5W_{t-1}$. Setting $W_0 = 0$:

```
wn_hat <- rep(0, n)
wn_hat[[1]] <- xt[[1]]
for (i in 2:n) {
  wn_hat[[i]] <- xt[[i]] - 0.5 * xt[[i - 1]] - 0.5 * wn_hat[[i - 1]]
}
```

- b. **Log absolute difference plot:**

```
tibble(t = 1:n,
       log_diff = log(abs(wn - wn_hat))) |>
  ggplot(aes(x = t, y = log_diff)) +
  geom_line() +
  labs(x = "Time", y = "log|W_t - W_hat_t|",
       title = "Log absolute difference between true and estimated white noise")
```

- c. From the plot, the log absolute difference decreases approximately **linearly** over time, which means the truncation error $|W_t - \hat{W}_t|$ decays **exponentially** fast. The rate of decay is governed by the MA coefficient $\theta = 0.5$: since $|\theta| < 1$, the initial approximation error ($W_0 = 0$) is multiplied by $(-\theta)^t = (-0.5)^t$ at each step, leading to exponential convergence.

This has an important consequence for truncated forecasts: even with the crude initialization $\hat{W}_0 = 0$, the estimated white noise values $\hat{W}_t$ become virtually indistinguishable from the true W_t after about 20–30 time steps. Therefore, the truncated forecast is an excellent approximation to the exact forecast for moderate to large n .

- d. **Truncated forecast for X_{53}** (i.e. $h = 3$ steps ahead from $n = 50$).

For the ARMA(1,1) model, the recursive forecast from time n is:

- $h = 1$: $\hat{X}_{51|50} = 0.5X_{50} + 0.5\hat{W}_{50}$
- $h = 2$: $\hat{X}_{52|50} = 0.5\hat{X}_{51|50}$ (since $E[W_{52}] = 0$ and $E[W_{51}] = 0$)
- $h = 3$: $\hat{X}_{53|50} = 0.5\hat{X}_{52|50}$

```
xhat_51 <- 0.5 * xt[n] + 0.5 * wn_hat[n]
xhat_52 <- 0.5 * xhat_51
xhat_53 <- 0.5 * xhat_52
cat("X_50 =", xt[n], "\n")
```

X_50 = 0.9860335

```
cat("W_hat_50 =", wn_hat[n], "\n")
```

W_hat_50 = 0.2811725

```
cat("X_hat_51 =", xhat_51, "\n")
```

X_hat_51 = 0.633603

```
cat("X_hat_52 =", xhat_52, "\n")
```

X_hat_52 = 0.3168015

```
cat("X_hat_53 =", xhat_53, "\n")
```

X_hat_53 = 0.1584008

Using $X_{50} \approx 0.986$ and $\hat{W}_{50} \approx 0.281$:

- $\hat{X}_{51|50} = 0.5(0.986) + 0.5(0.281) \approx 0.634$
- $\hat{X}_{52|50} = 0.5(0.634) \approx 0.317$
- $\hat{X}_{53|50} = 0.5(0.317) \approx 0.158$

The forecasts decay geometrically toward 0 (the process mean) at rate 0.5^h , since for $h \geq 2$ the MA component no longer contributes and the forecast follows the pure AR(1) recursion.

6. ACF, PACF, and BLPs

Let (X_t) be a mean zero stationary process with $\gamma_X(0) = 2$, $\gamma_X(1) = 1.4$, $\gamma_X(2) = 0.6$, $\gamma_X(3) = 0.4$, $\gamma_X(4) = 0.2$.

The autocorrelation function is $\rho(h) = \gamma(h)/\gamma(0)$:

```
gamma_x <- c(2, 1.4, 0.6, 0.4, 0.2)
rho <- gamma_x / gamma_x[1]
names(rho) <- paste0("rho(", 0:4, ")")
rho
```

rho(0)	rho(1)	rho(2)	rho(3)	rho(4)
1.0	0.7	0.3	0.2	0.1

a. **Can (X_t) be an MA(2) process?**

No. For an MA(2) process, $\gamma(h) = 0$ for all $|h| > 2$. Since $\gamma(3) = 0.4 \neq 0$, (X_t) cannot be MA(2).

b. **Can (X_t) be an AR(1) process?**

For AR(1), $\gamma(h) = \phi^h \gamma(0)$. From $\gamma(1)/\gamma(0) = 0.7$, we would need $\phi = 0.7$. Then $\gamma(2)$ should equal $\phi^2 \gamma(0) = 0.49 \times 2 = 0.98$. But $\gamma(2) = 0.6 \neq 0.98$, so (X_t) **cannot** be AR(1).

c. **BLP of X_4 given $X_3 = 2$:**

For a mean zero process with a single predictor, $\hat{X}_4 = \frac{\text{Cov}(X_4, X_3)}{\text{Var}(X_3)} X_3 = \frac{\gamma(1)}{\gamma(0)} X_3$.

```
xhat_4c <- (gamma_x[2] / gamma_x[1]) * 2
cat("BLP of X_4 given X_3 = 2:", xhat_4c, "\n")
```

BLP of X_4 given $X_3 = 2$: 1.4

d. **Variance of $X_4 - \hat{X}_4$ (from part c):**

$$\text{Var}(X_4 - \hat{X}_4) = \gamma(0) - \frac{\gamma(1)^2}{\gamma(0)} = 2 - \frac{1.96}{2} = 1.02$$

```
var_4c <- gamma_x[1] - gamma_x[2]^2 / gamma_x[1]
cat("Variance:", var_4c, "\n")
```

Variance: 1.02

e. **BLP of X_4 given $X_2 = 2$:**

$$\hat{X}_4 = \frac{\text{Cov}(X_4, X_2)}{\text{Var}(X_2)} X_2 = \frac{\gamma(2)}{\gamma(0)} X_2$$

```
xhat_4e <- (gamma_x[3] / gamma_x[1]) * 2
cat("BLP of X_4 given X_2 = 2:", xhat_4e, "\n")
```

BLP of X_4 given $X_2 = 2$: 0.6

f. **Variance of $X_4 - \hat{X}_4$** (from part e):

$$\text{Var}(X_4 - \hat{X}_4) = \gamma(0) - \frac{\gamma(2)^2}{\gamma(0)} = 2 - \frac{0.36}{2} = 1.82$$

```
var_4f <- gamma_x[1] - gamma_x[3]^2 / gamma_x[1]
cat("Variance:", var_4f, "\n")
```

Variance: 1.82

g. **PACF $\alpha_X(1)$:**

The PACF at lag 1 equals the ACF at lag 1:

$$\alpha_X(1) = \rho(1) = \frac{\gamma(1)}{\gamma(0)} = \frac{1.4}{2} = 0.7.$$

h. **PACF $\alpha_X(3)$:**

$\alpha_X(3)$ is the last coefficient ϕ_3 when solving the Yule-Walker system $\Gamma_3\phi = \rho_{1:3}$ for an AR(3) model:

```
Gamma3 <- matrix(c(1, rho[2], rho[3],
                  rho[2], 1, rho[2],
                  rho[3], rho[2], 1), nrow = 3, byrow = TRUE)
phi3 <- solve(Gamma3, rho[2:4])
names(phi3) <- paste0("phi_", 1:3)
phi3
```

```
      phi_1      phi_2      phi_3
1.1071429 -0.7500000  0.3928571
```

```
cat("alpha_X(3) =", phi3[3], "\n")
```

alpha_X(3) = 0.3928571

From the Yule-Walker solution, the AR(3) coefficients are $\hat{\phi}_1 \approx 1.107$, $\hat{\phi}_2 \approx -0.750$, $\hat{\phi}_3 \approx 0.393$. The PACF at lag 3 is $\alpha_X(3) = \hat{\phi}_3 \approx 0.393$, which is nonzero. This further confirms that the process is not AR(1) or AR(2), since for an AR(p) process, $\alpha_X(h) = 0$ for all $h > p$.